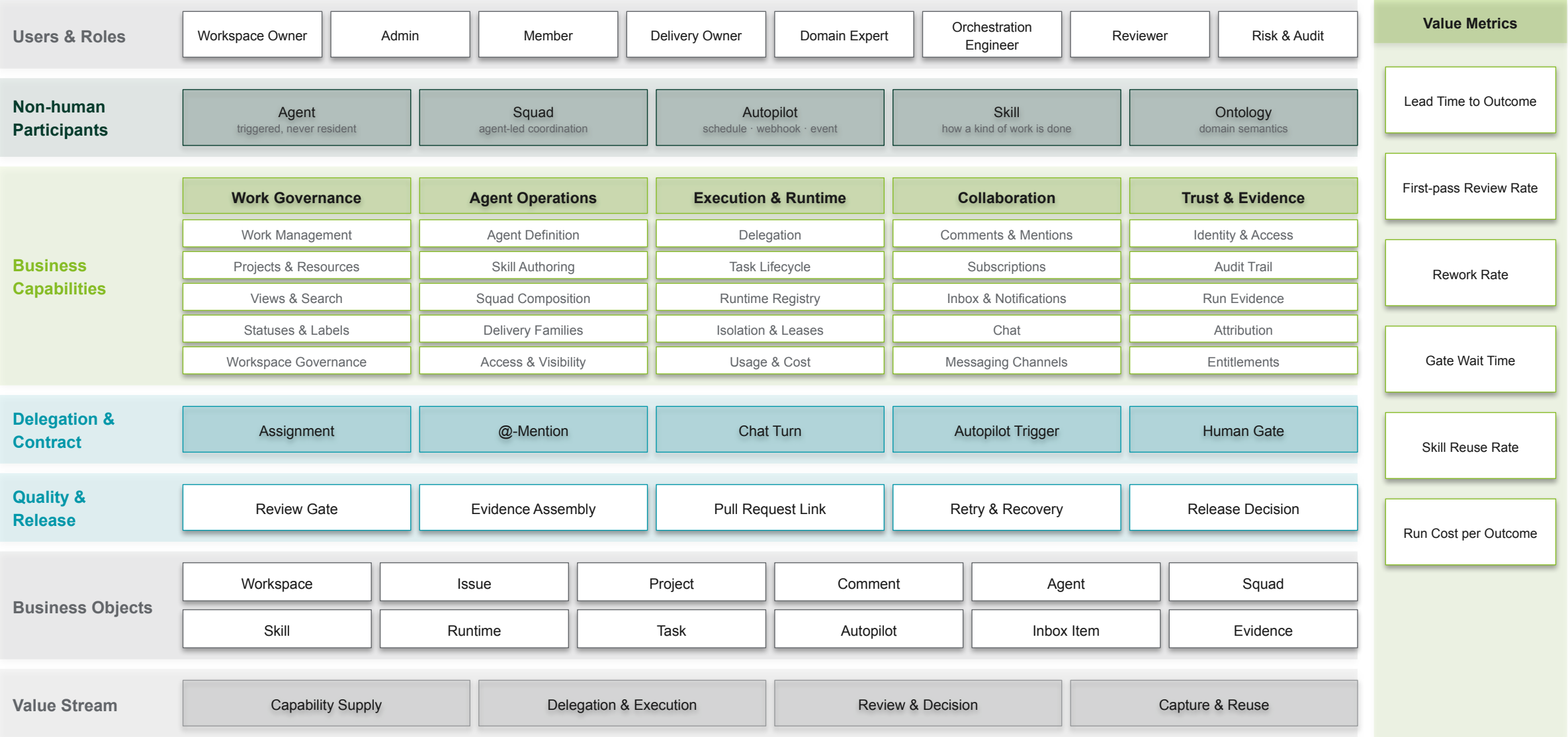
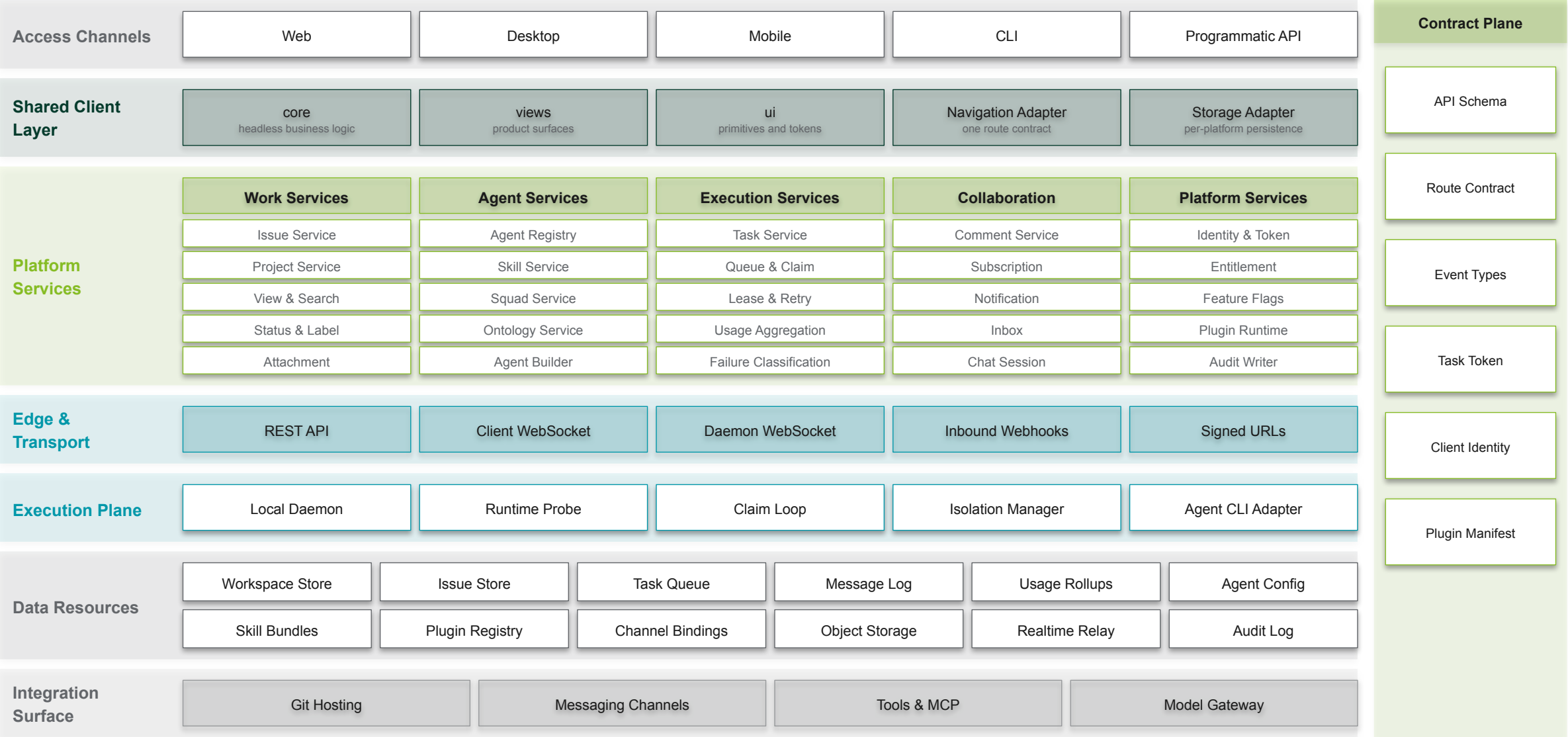


Business Architecture



Application Architecture



■ Technical Architecture

Client Tier	Next.js 16 · App Router React 19.2 · webpack build	Electron 39 · electron-vite memory router · contextBridge · sandbox	Expo 55 · React Native 0.83 iOS · NativeWind · its own query client	Go CLI + daemon Cobra · single binary · self-update	
Access & Security	JWT cookie + CSRF HttpOnly · SameSite=Strict	enact_ PAT · mat_ task token task token expires with the run	Workspace filter · Access every query scoped; admins cannot bypass	secretbox · OS-user boundary sealed integration keys	
Frontend Runtime & Protocols	TypeScript strict raw .ts packages, transpiled by consumers	TanStack Query v5 · Zustand v5 server state vs client state	zod · parseWithFallback network JSON is never cast	WebSocket · client + daemon scope subscribe · WS-first RPC	REST · webhooks · MCP · VCS HMAC-verified ingress
	API server	Task engine	Realtime hub	Schedulers & workers	Execution daemon
	Chi router + middleware	agent_task_queue	Per-workspace rooms	sys_cron_executions lease	Runtime probe
	sqlc · pgx/v5 · pgxpool	Batch claim + lease	Scope subscriptions	Usage rollup	Claim loop · WS + poll
	Workspace-scoped queries	5-minute prepare bound	Redis Streams relay	Autopilot dispatch	execenv re-exec · setpgid
	slog + X-Request-ID	Retry · backoff · classify	XREAD BLOCK + replay	Webhook delivery	git worktree isolation
Go Application Runtime	Recoverer · CORS · CSP	Orphan recovery	Slow-client eviction	Channel supervisor	23 agent CLI backends
Data & Platform Services	PostgreSQL 17 435 migrations · advisory-locked runner	No foreign keys integrity resolved in application code	Concurrent index builds one statement per migration file	pg_bigm + pg_trgm GIN CJK search · 3s timeout · 57014 → 503	Redis · optional relay · rate limit · claim cache; fails open
Operations & Infrastructure	Docker Compose · Helm entrypoint migrates, then serves	Prometheus on METRICS_ADDR separate server · own sampler pool	slog + tint · pprof X-Request-ID correlation · loopback only	S3 + CloudFront · workdir presigned GET · signed cookies	GoReleaser · GHCR CLI archives · multi-arch images

External Resources

GitHub · GitLab
Gitea · Forgejo

Slack · Lark · WeCom
DingTalk · Telegram

MCP servers · Composio

AI provider CLIs

Stripe via Cloud · PostHog

The agent outlives the tool it runs on

Teams change coding agents every few months. Enact is built so that changing the tool does not mean rebuilding the team, its permissions or the audit trail.

01

Identity, capability and compute are separate

An agent is a configuration — charter, instructions, skills, Access. A runtime is the machine and the tool that runs it. Swapping either leaves the other intact.

23 protocol families are supported, and the whitelist is enforced twice — in Go and as a database constraint — so a workspace can only build on a backend Enact actually supports.

02

Tool access resolves at dispatch, not at setup

Workspace, agent, per-run and plugin-bridged MCP servers merge by name. On collision the live user credential wins over the saved placeholder.

A malformed overlay ships the saved servers unchanged — silently disabling a team's existing tools is the worse failure.

03

Scoped by capability, never by trust

Plugins declare scopes from a closed vocabulary. `net:<domain>` is not advisory — it drives the iframe CSP allowlist and the hook host check.

MCP approvals are pinned to a schema digest, so a tool that changes what it does needs fresh consent. Remote credentials are fetched per connection, never stored in a task record.

Nothing here assumes the tool you use today is the tool you will use next year.

Eight stages an agent can run. Five decisions it may not make.

The delivery method ships as ten executable skills rather than a diagram. Every new workspace is provisioned with nine system agents and a delivery squad.

01

The method is executable, not documentary

Intake, Explore, Contract, Build, QA, Release, Operate, Learn — each a skill with a named exit artefact. sdlc-core holds the shared object model, state machine and validators.

State lives as files under .sdlc/ — git-diffable, reviewable in a pull request, auditable offline. No database, no session memory.

02

Five gates need a named human

contract-approval, scope-expansion, build-review, release, lesson-approval. Each is two-phase: deterministic pre-check first, named authorisation second.

A model gathers evidence, explains it and recommends. It does not sign. An agent, the workspace owner, silence or a status change is never approval.

03

Separation of duties is enforced

QA freezes test intent from the approved contract before reading the implementation. Build may not fill a contract gap in code — it raises an amendment.

Rollback is a first-class path, including the exit that lets “this should not be built” be recorded as exactly that.

Ceremony scales with the task. Approval gates never do.

The boundary you can verify beats the sandbox you were promised

Enact does not claim a filesystem sandbox. It states where the boundary actually sits and gives you three ways to place it — a stronger position in a security review than a claim that does not hold.

01

Authorisation narrows four times

Every query filters by workspace. Organisation roles govern settings, not resource visibility. Per-resource Access binds invocation — an admin may manage an agent and still not be allowed to run it.

Sensitive endpoints require a human principal, so an agent carrying its owner's identity still cannot reach that owner's account.

02

Credentials are scoped to the run

A task token is bound to user, agent, task and workspace, and expires when the run does.

Integration secrets are sealed per integration, and plugin config has a dedicated secret type so a plugin never renders its own credential form.

03

The real boundary is the OS user

Agents run unattended, so approval prompts auto-answer and the tool's own sandbox is off. Linux workspace-write was removed: restricting writes only, it never stopped a task exfiltrating credentials.

Place the boundary yourself — a dedicated system user, a container, or a virtual machine.

Per-task worktrees and scoped tokens shrink the blast radius. They are not an adversarial boundary, and we do not sell them as one.

Nothing becomes a standard by accident

Two mechanisms carry knowledge: agents read enterprise fact through a read-only surface and propose changes back as ordinary git, and validated experience is promoted only by a named decision.

01

Agents read the facts, they do not hold a copy

Intelligence Space keeps one source with three views — the site, a committed machine export and an MCP surface — all fed by one loader, so they cannot describe different content.

Eight read-only tools cover catalogue, artefact, document, search and traceability. The server re-reads on change, so an agent sees the working tree, never a stale snapshot.

02

Write-back is a branch and a merge

A proposal is a git branch in its own worktree. The pre-commit hook regenerates the export and validates, so content the site cannot render — or an export that has drifted — cannot be committed.

Approving merges with `--no-ff` and removes the branch; rejecting leaves the commits recoverable in the reflog.

03

Experience is promoted by decision

Three memory tiers decide what may be cited as fact — they stop an unverified lesson becoming standard by being quoted as “last time we hit X” in the next piece of work.

Promotion runs six stages, classified by blast radius from one project up to every project, and approved by a named steward.

Facts flow in through a read-only surface. Changes flow back as reviewed commits. Nothing is promoted without someone deciding.